K-MEANS CLUSTERING

1. Konsep dan Pengertian

K-Means Clustering adalah algoritma **Unsupervised Learning** yang digunakan untuk mengelompokkan data ke dalam sejumlah K kelompok (cluster) berdasarkan kemiripan karakteristik atau jarak antar data.

Algoritma ini bekerja dengan menentukan K centroid (titik pusat cluster) -> menghitung jarak setiap data terhadap seluruh centroid -> memasukkan data ke cluster dengan centroid terdekat.

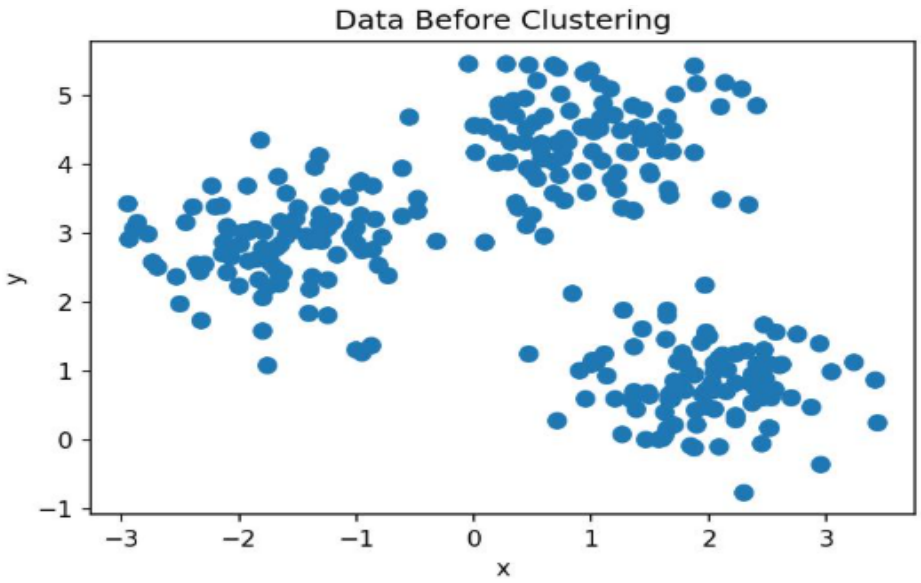
Setelah seluruh data dikelompokkan, posisi centroid diperbarui berdasarkan rata-rata seluruh data dalam cluster tersebut. Proses pengelompokan dan pembaruan centroid dilakukan secara berulang hingga posisi centroid tidak lagi berubah secara signifikan atau algoritma mencapai batas iterasi.

Tujuan utama K-Means adalah meminimalkan variasi jarak data terhadap centroid dalam masing-masing cluster, sehingga data yang memiliki karakteristik serupa berada dalam kelompok yang sama.

2. Visualisasi Algoritma K-Means

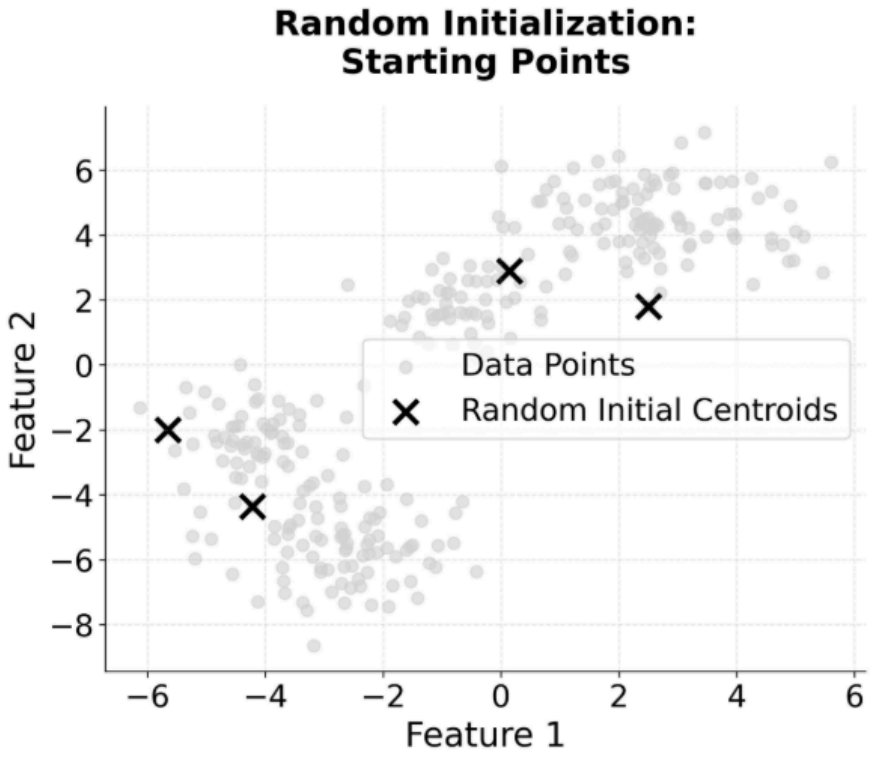
K-Means bekerja melalui proses iterasi. Berikut gambaran prosesnya dari data awal sampai terbentuk cluster.

Tahap 1 - Data Awal



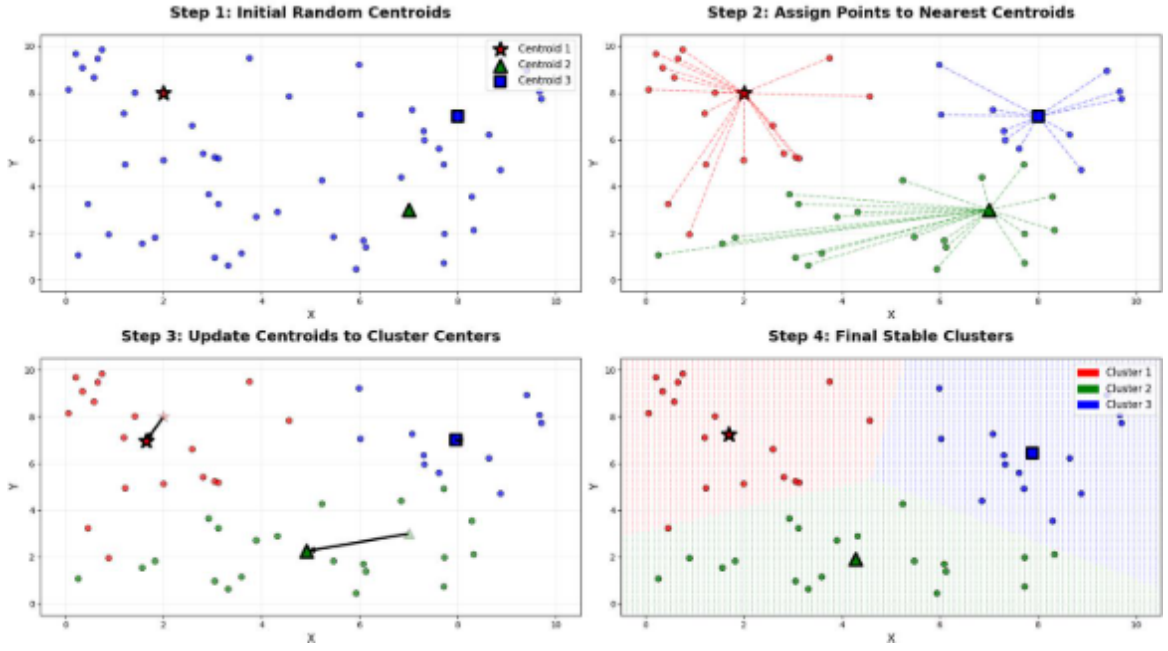
Data masih belum memiliki label cluster. Algoritma hanya menerima nilai fitur dari setiap data.

Tahap 2 - Inisialisasi Centroid



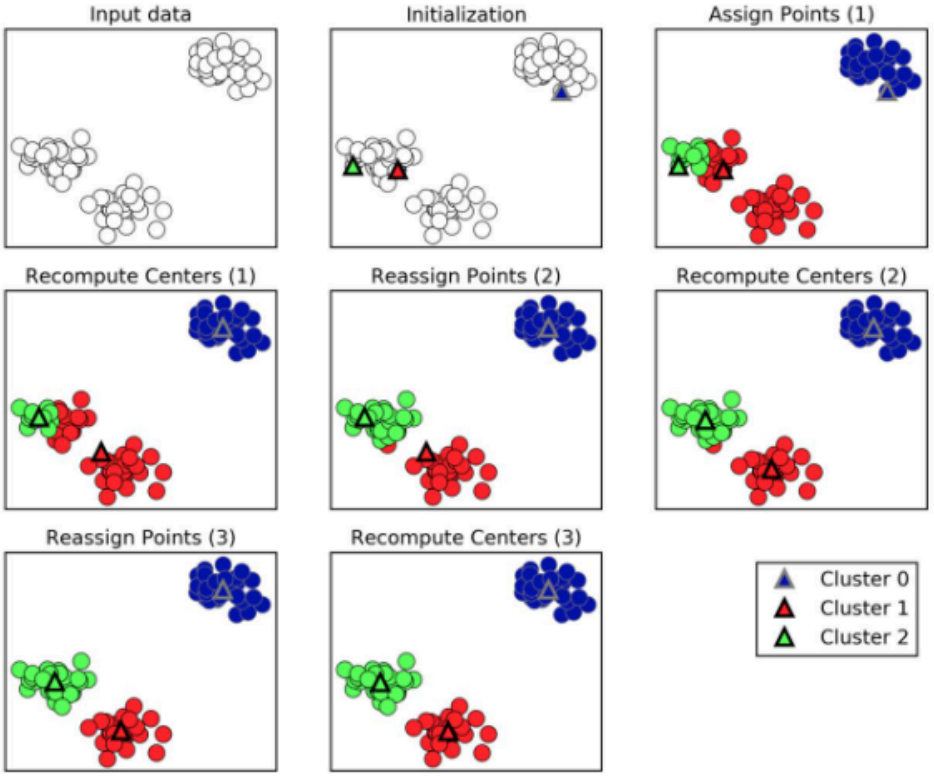
Algoritma menentukan K titik pusat awal yang disebut centroid.

Tahap 3 - Assignment (Pengelompokan)



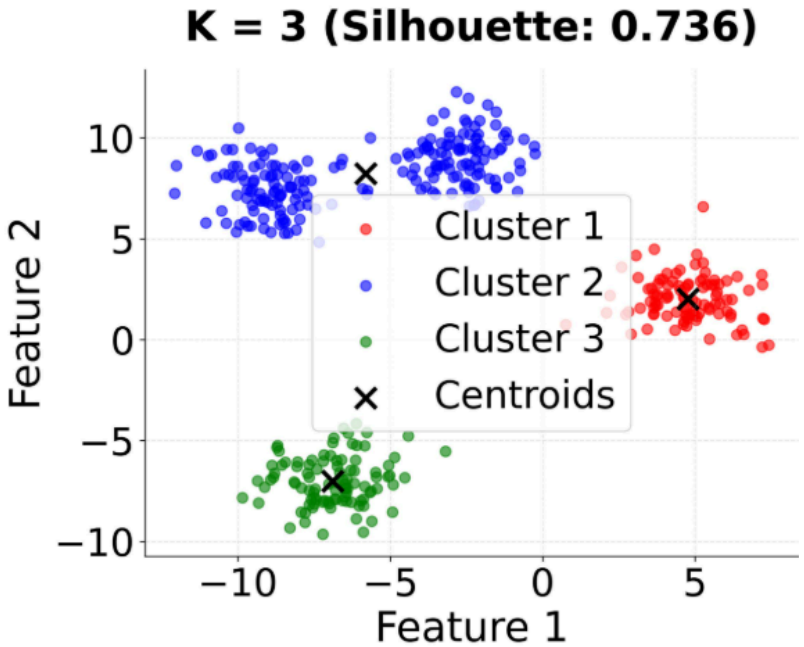
Setiap data dihitung jaraknya ke seluruh centroid. Data dimasukkan ke cluster dengan jarak paling kecil.

Tahap 4 - Update Centroid



Centroid dihitung ulang menggunakan rata-rata fitur seluruh anggota cluster.

Tahap 5 - Hasil Akhir



3. Analogi Sederhana

Bayangkan terdapat 30 siswa di sebuah lapangan dan kamu ingin membaginya menjadi 3 kelompok berdasarkan tinggi badan dan berat badan.

Kamu memilih 3 siswa sebagai perwakilan awal masing-masing kelompok. Kemudian, setiap siswa lainnya akan bergabung dengan kelompok yang perwakilannya paling mirip berdasarkan tinggi dan berat badan.

Setelah semua siswa masuk ke kelompok, kamu menghitung rata-rata tinggi dan berat badan setiap kelompok. Nilai rata-rata tersebut menjadi posisi perwakilan baru.

Selanjutnya, siswa diperiksa kembali. Bisa saja ada siswa yang sekarang lebih dekat dengan kelompok lain karena posisi perwakilannya berubah.

Proses tersebut terus dilakukan sampai pembagian kelompok stabil.

Pemetaan analogi ke algoritma:

Analogi	K-Means
Jumlah kelompok	K
Perwakilan kelompok	Centroid
Tinggi dan berat badan	Features
Kemiripan siswa	Jarak antar data
Memasukkan siswa ke kelompok	Assignment
Menghitung rata-rata kelompok	Update centroid
Kelompok sudah stabil	Convergence

4. Jenis-Jenis dan Variasi K-Means

Jenis	Penjelasan	Kegunaan
Standard K-Means	Menggunakan centroid awal yang dipilih secara acak	Clustering umum
K-Means++	Memilih centroid awal agar tersebar lebih baik	Mengurangi inisialisasi buruk

Pendekatan penentuan jumlah K

Jumlah cluster tidak otomatis diketahui. Beberapa pendekatan yang sering digunakan:

1. **Elbow Method**: Mencari titik siku pada grafik WCSS untuk memperkirakan jumlah cluster yang sesuai.
2. **Silhouette Score**: Mengukur seberapa baik suatu data berada di dalam clusternya dibandingkan dengan cluster lain.
3. **Domain Knowledge**: Menentukan jumlah cluster berdasarkan kebutuhan atau pengetahuan terhadap masalah.

1. Kelebihan K-Means

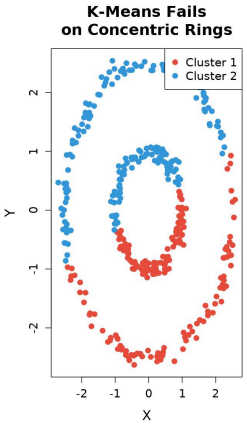
No.	Kelebihan	Penjelasan
1	Mudah dipahami	Konsepnya sederhana, yaitu mengelompokkan data berdasarkan centroid terdekat.
2	Proses relatif cepat	Efisien untuk dataset berukuran besar dibandingkan beberapa algoritma clustering lainnya.

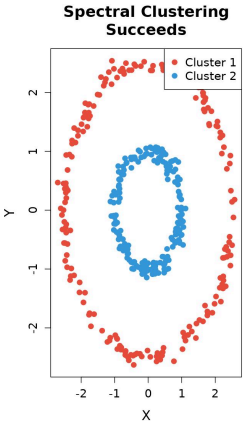
3	Mudah diimplementasikan	Bisa menggunakan Scikit-learn dengan kode yang relatif singkat.
4	Mudah dikembangkan	Bisa digunakan untuk berbagai kebutuhan segmentasi data.
5	Hasil mudah divisualisasikan	Cluster dapat ditampilkan menggunakan scatter plot jika memiliki dua atau tiga fitur.
6	Relatif hemat memori	Tidak perlu menyimpan seluruh matriks jarak antar-data seperti beberapa metode clustering lainnya.

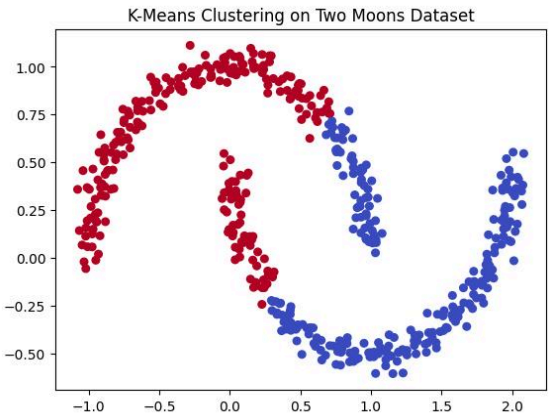
2. Kekurangan K-Means dan Cara Mengatasinya

No.	Kekurangan	Penjelasan	Solusi
1	Harus menentukan K	Jumlah cluster harus ditentukan sebelum algoritma dijalankan.	Gunakan Elbow Method atau Silhouette Score.
2	Sensitif terhadap outlier	Data ekstrem dapat menarik posisi centroid.	Deteksi dan tangani outlier sebelum clustering.
3	Sensitif terhadap inisialisasi centroid	Titik awal centroid dapat memengaruhi hasil akhir.	Gunakan K-Means++ dan beberapa inisialisasi (`n_init`).
4	Sensitif terhadap skala fitur	Fitur dengan rentang besar dapat mendominasi perhitungan jarak.	Gunakan StandardScaler atau MinMaxScaler sesuai karakteristik data.

5	Kurang cocok untuk cluster dengan bentuk kompleks	K-Means lebih cocok untuk cluster yang relatif bulat atau kompak.	Pertimbangkan DBSCAN atau algoritma clustering lain.
6	Hanya cocok untuk data numerik	Perhitungan centroid menggunakan nilai rata-rata.	Lakukan encoding yang sesuai atau gunakan algoritma lain untuk data kategorikal.
7	Bisa terjebak pada solusi lokal	Hasil akhir belum tentu merupakan solusi global terbaik.	Gunakan beberapa inisialisasi dan bandingkan hasilnya.







Kondisi Dataset	Kesesuaian
Data numerik dengan cluster yang relatif bulat	Cocok
Data pelanggan dengan fitur pengeluaran dan frekuensi belanja	Cocok
Data memiliki banyak outlier ekstrem	Kurang cocok

Cluster berbentuk bulan sabit	Kurang cocok
Data kategorikal murni	Kurang cocok

5. Alur Pengerjaan K-Means Clustering

Data	X1 (Pembelian)	X2 (Kunjungan)
A	1	1
B	2	2
C	8	8
D	9	9

Tentukan:

- Jumlah cluster: K = 2.
- Centroid awal C1 = (1,1).
- Centroid awal C2 = (8,8).
- Metode jarak: Euclidean Distance.

Langkah 1 - Menghitung Euclidean Distance

Rumus:

$$d = \sqrt{(x_1 - c_1)^2 + (x_2 - c_2)^2}$$

Kita menghitung jarak setiap data terhadap kedua centroid.

Data A = (1,1)

Jarak ke C1 = (1,1):

$$d(A, C1) = \sqrt{(1 - 1)^2 + (1 - 1)^2}$$

$$= \sqrt{0 + 0} = 0$$

Jarak ke C2 = (8,8):

$$d(A, C2) = \sqrt{(1 - 8)^2 + (1 - 8)^2}$$

$$= \sqrt{49 + 49} = 9.899$$

Maka A masuk Cluster 1 karena jaraknya lebih dekat ke C1.

Data B = (2,2)

Jarak ke C1:

$$d(B, C1) = \sqrt{(2 - 1)^2 + (2 - 1)^2}$$

$$= \sqrt{2} = 1.414$$

Jarak ke C2:

$$d(B, C2) = \sqrt{(2 - 8)^2 + (2 - 8)^2}$$

$$= \sqrt{72} = 8.485$$

Maka B masuk Cluster 1.

Data C = (8,8)

Jarak ke C1:

$$d(C, C1) = \sqrt{(8 - 1)^2 + (8 - 1)^2}$$

$$= \sqrt{98} = 9.899$$

Jarak ke C2:

$$d(C, C2) = 0$$

Maka C masuk Cluster 2.

Data D = (9,9)

Jarak ke C1:

$$d(D, C1) = \sqrt{(9 - 1)^2 + (9 - 1)^2}$$

$$= \sqrt{128} = 11.314$$

Jarak ke C2:

$$d(D, C2) = \sqrt{(9 - 8)^2 + (9 - 8)^2}$$

$$= \sqrt{2} = 1.414$$

Maka D masuk Cluster 2.

Hasil Assignment Pertama

Data	Jarak C1	Jarak C2	Cluster
A (1,1)	0.000	9.899	1

B (2,2)	1.414	8.485	1
C (8,8)	9.899	0.000	2
D (9,9)	11.314	1.414	2

- Cluster 1 = A, B
- Cluster 2 = C, D

Langkah 2 - Menghitung Centroid Baru

Rumus:

$$C_k = \left(\frac{\sum X1}{N}, \frac{\sum x2}{N} \right)$$

Centroid baru Cluster 1

Anggota: A (1,1) dan B (2,2).

$$C1_{x1} = \frac{1+2}{2} = 1.5$$

$$C1_{x2} = \frac{1+2}{2} = 1.5$$

Centroid baru Cluster 2

Anggota: C (8,8) dan D (9,9).

$$C2_{x1} = \frac{8+9}{2} = 8.5$$

$$C2_{x2} = \frac{8+9}{2} = 8.5$$

Langkah 3 - Iterasi Kedua

Sekarang kita hitung kembali jarak menggunakan centroid baru.

Data	**Jarak C1 (1.5,1.5)**	**Jarak C2 (8.5,8.5)**	**Cluster**
A (1,1)	0.707	10.607	1
B (2,2)	0.707	9.192	1
C (8,8)	9.192	0.707	2
D (9,9)	10.607	0.707	2

Perhatikan bahwa seluruh data masih berada di cluster yang sama.

Artinya:

- Anggota Cluster 1 tidak berubah.
- Anggota Cluster 2 tidak berubah.
- Centroid baru juga tetap sama.

Langkah 4 - Menghitung WCSS

Menghitung seberapa rapat jarak data di masing” centroid, dengan centroid itu sendiri.Semakin kecil nilai wcss, maka semakin baik.

Rumus:

$$WCSS = \sum_{k=1}^{K} \sum_{x_i \in C_k} \|x_i - \mu_k\|^2$$

Cluster 1:

Centroid = (1.5,1.5)

Data A:

$$(1 - 1.5)^2 + (1 - 1.5)^2 = 0.5$$

Data B:

$$(2 - 1.5)^2 + (2 - 1.5)^2 = 0.5$$

$$WCSS_1 = 0.5 + 0.5 = 1$$

Cluster 2:

Centroid = (8.5,8.5)

Data C:

$$(8 - 8.5)^2 + (8 - 8.5)^2 = 0.5$$

Data D:

$$(9 - 8.5)^2 + (9 - 8.5)^2 = 0.5$$

$$WCSS_2 = 0.5 + 0.5 = 1$$

Total:

$$WCSS = WCSS_1 + WCSS_2$$

WCSS = 2

Ingat tiga proses inti K-Means:

1. **Distance**: Menghitung jarak data ke centroid.
2. **Assignment**: Memasukkan data ke cluster terdekat.
3. **Update**: Menghitung centroid baru menggunakan rata-rata.

Ketiga proses tersebut diulang sampai konvergen.

6. Implementasi ke Python

```python
# FUNGSI K-MEANS
def hitung_kmeans(data, k, max_iter=100):
    # Centroid awal acak
    idx = np.random.choice(len(data), k, replace=False)
    centroids = data[idx]

    for _ in range(max_iter):
        # Hitung jarak Euclidean
        dist = np.sqrt(((data - centroids[:, np.newaxis])**2).sum(axis=2))
        labels = np.argmin(dist, axis=0)

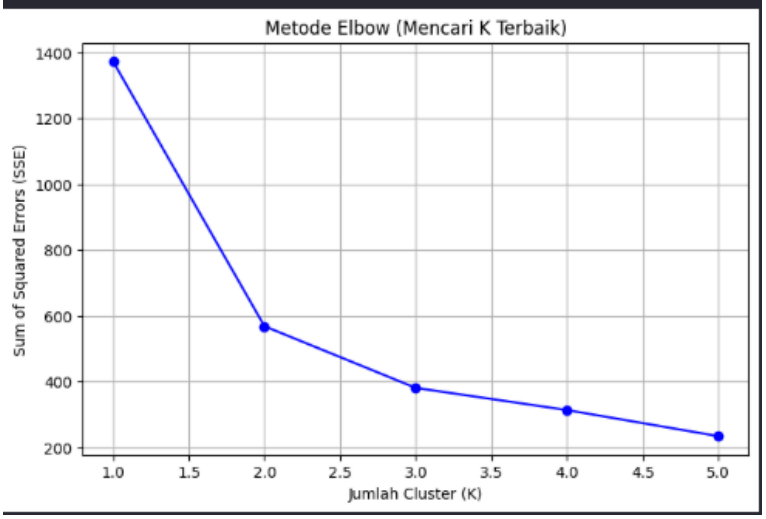
        # Update Centroid
        new_centroids = np.array([data[labels == i].mean(axis=0) if len(data[labels == i]) > 0
                                  else centroids[i] for i in range(k)])
        if np.allclose(centroids, new_centroids):
            break
        centroids = new_centroids

    # Hitung SSE
    sse = 0
    for i in range(k):
        cluster_points = data[labels == i]
        sse += np.sum((cluster_points - centroids[i])**2)
    return centroids, labels, sse
```

Visualisasi SSE

```python
sse_list = []
ks = range(1, 6)
for k in ks:
    _, _, sse_val = hitung_kmeans(scaled_data, k)
    sse_list.append(sse_val)

plt.figure(figsize=(8, 5))
plt.plot(ks, sse_list, 'bo-')
plt.title('Metode Elbow (Mencari K Terbaik)')
plt.xlabel('Jumlah Cluster (K)')
plt.ylabel('Sum of Squared Errors (SSE)')
plt.grid(True)
plt.show()
```



6. Rumus Matematika K-Means Clustering

Ada beberapa rumus penting yang wajib dipahami untuk menguasai K-Means.

A. Euclidean Distance

(Rumus jarak utama)

Digunakan untuk menghitung jarak antara data dengan centroid.

$$d(x, c) = \sqrt{\sum_{j=1}^{n} (x_j - c_j)^2}$$

Keterangan:

- d = jarak.
- x_j = nilai fitur data.
- c_j = nilai fitur centroid.
- n = jumlah fitur.
- j = indeks fitur.

Untuk dua dimensi:

$$d = \sqrt{(x_1 - c_1)^2 + (x_2 - c_2)^2}$$

Contoh:

Data = (3, 4)

Centroid = (1, 1)

$$d = \sqrt{(3 - 1)^2 + (4 - 1)^2}$$

$$d = \sqrt{4 + 9} = \sqrt{13}$$

$$d = 3.606$$

Fungsi: Menentukan centroid mana yang paling dekat dengan suatu data.

B. Centroid Update (Mean)

Digunakan untuk menghitung posisi centroid baru berdasarkan rata-rata seluruh anggota cluster.

$$C_k = \frac{1}{N_k} \sum_{i=1}^{N_k} x_i$$

Keterangan:

- C_k = centroid cluster ke-k.
- N_k = jumlah data dalam cluster.
- x_i = data anggota cluster.

Contoh:

Data anggota cluster:

$$(2, 4), (4, 6), (6, 8)$$

Centroid baru:

$$C = \left(\frac{2+4+6}{3}, \frac{4+6+8}{3}\right)$$

$$C = (4, 6)$$

Fungsi: Memindahkan pusat cluster ke posisi rata-rata anggotanya.

C. Within-Cluster Sum of Squares (WCSS)

WCSS merupakan fungsi objektif yang ingin diminimalkan oleh K-Means.

$$WCSS = \sum_{k=1}^{K} \sum_{x_i \in C_k} \|x_i - \mu_k\|^2$$

Keterangan:

- K = jumlah cluster.
- C_k = anggota cluster ke-k.
- x_i = data.
- μ_k = centroid cluster.
-

Fungsi: Mengukur total variasi atau kesalahan kuadrat jarak data terhadap centroid dalam setiap cluster.

Semakin kecil WCSS, semakin dekat data dengan centroid masing-masing. Namun, WCSS akan selalu cenderung turun ketika jumlah K bertambah. Karena itu, WCSS tidak boleh digunakan sendirian untuk menentukan K terbaik.

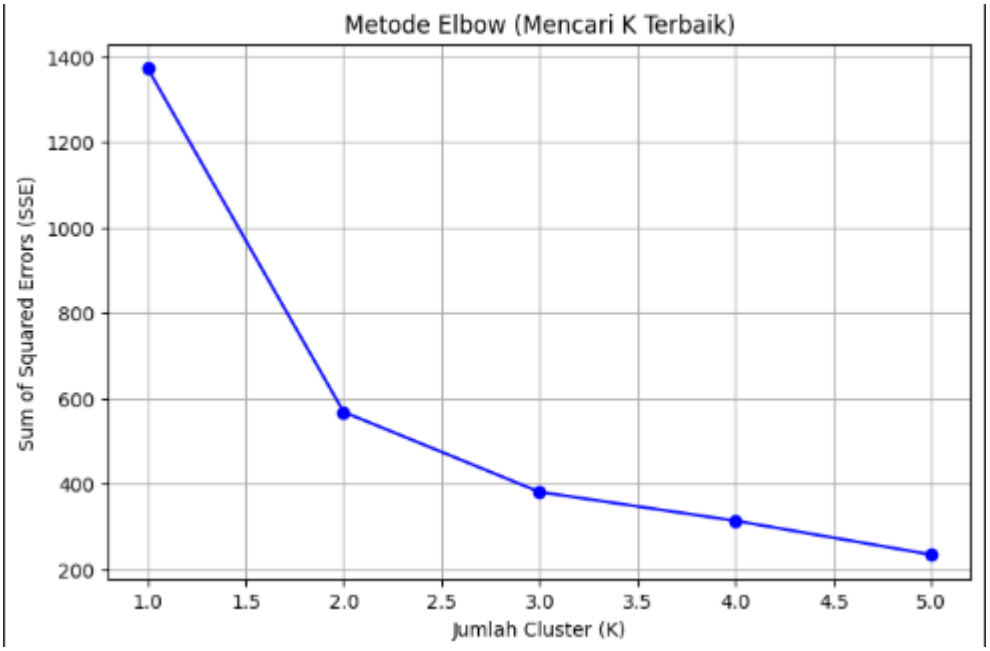
D. Elbow Method

Digunakan untuk memperkirakan jumlah cluster yang sesuai dengan melihat perubahan WCSS ketika nilai K meningkat.

Tidak memiliki satu rumus khusus yang berbeda dari WCSS. Prosesnya adalah menghitung WCSS untuk beberapa nilai K.

$$K = 1, 2, 3, \ldots, n$$

Kemudian membuat grafik K terhadap WCSS.



Cara membaca:

- Sumbu X = jumlah cluster (K).
- Sumbu Y = WCSS.
- Pilih titik ketika penurunan WCSS mulai melambat secara signifikan.

Titik tersebut disebut elbow atau siku.

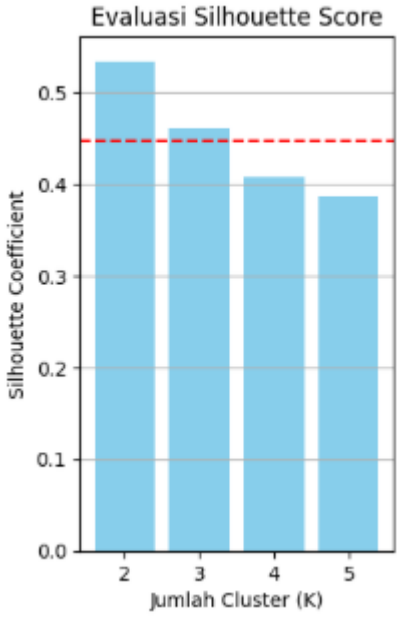
E. Silhouette Score

Digunakan untuk mengukur seberapa baik suatu data berada di cluster-nya dibandingkan dengan cluster lain.

$$S(i) = \frac{b(i)-a(i)}{\max(a(i),b(i))}$$

Keterangan:

- $a(i)$ = rata-rata jarak data ke anggota cluster sendiri.
- $b(i)$ = rata-rata jarak terendah ke anggota cluster lain.
- $S(i)$ = nilai silhouette data tersebut.

- 

Rentang nilai:

Nilai	Interpretasi
Mendekati +1	Cluster sudah baik dan terpisah
Mendekati 0	Data berada di perbatasan cluster
Mendekati -1	Data kemungkinan salah penempatan cluster

Fungsi: Mengevaluasi kualitas hasil clustering dengan mempertimbangkan kekompakan dan pemisahan cluster.

Ringkasan rumus

Rumus	Fungsi
Euclidean Distance	Menghitung jarak data ke centroid
Centroid Update	Menghitung pusat cluster baru
WCSS	Mengukur total kesalahan kuadrat dalam cluster
Elbow Method	Memperkirakan jumlah cluster
Silhouette Score	Mengevaluasi kualitas cluster

Visualisasi Evaluasi Akhir

